

OptiCrop: Coding & Solution

Backend (Flask) - app.py

```
model = pickle.load(open('model.pkl', 'rb'))
scaler = pickle.load(open('scaler.pkl', 'rb'))
label_encoder = pickle.load(open('label_encoder.pkl', 'rb'))

@app.route('/predict', methods=['POST'])
def predict():
    N = float(request.form['N'])
    ...
    input_data = pd.DataFrame([[N, P, K, temperature, humidity, ph, rainfall]],
                              columns=FEATURE_NAMES)
    input_scaled = scaler.transform(input_data)
    prediction = model.predict(input_scaled)
    predicted_crop = label_encoder.inverse_transform(prediction)[0]
    return render_template('result.html', crop=predicted_crop)
```

Model Training (Notebook) - Key Steps

- Load and clean Crop_recommendation.csv; handle outliers via the IQR method.
- Scale numeric features with StandardScaler; encode crop labels with LabelEncoder.
- Split into train/test sets; train a Logistic Regression classifier.
- Evaluate using accuracy, precision, recall, F1-score, and ROC-AUC.
- Run K-Means clustering (elbow method to choose k=4) on the same feature set.
- Serialize model, scaler, and label encoder with pickle for use by the Flask app.

Result

A working end-to-end pipeline: raw CSV data in, trained model artifacts out, served live through a Flask web form.